DÉMO 004-F · CHANGEMENTS ENTRE LES ÉTAPES

# Cycle de vie et optimisations

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-cycle-logs (3 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-react-memo (1 fichier)
3. **Étape 2 → 3** · etape-3-props-recreees (1 fichier)
4. **Étape 3 → 4** · etape-4-usecallback-usememo (1 fichier)

## Étape 1 etape-1-cycle-logs : point de départ

Cycle de vie : renderCount avec useRef, logs dans useEffect (Mounted / Unmounted)

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

**Styles fournis (copiés tels quels, pas tapés en live)** : src/App.css (48 lignes)

**Fichiers générés par Vite, non modifiés (non affichés)** : index.html, vite.config.js, eslint.config.js, public/vite.svg, package.json

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 16 lignes

```jsx
import { useState } from "react";
import Cars from "./components/Cars";
import "./App.css";

export default function App() {
  const [showHome, setShowHome] = useState(true);

  return (
    <div>
      <button onClick={() => setShowHome((v) => !v)}>
        {showHome ? "Unmount Home" : "Mount Home"}
      </button>
      {showHome ? <Cars /> : <div>Home is hidden</div>}
    </div>
  );
}
```

DÉPART **src/components/Cars.jsx** 96 lignes

```jsx
import { useEffect, useRef, useState } from "react";

function Car({ make, color }) {
  // useRef garde le compteur d'un render à l'autre, sans provoquer de nouveau render
  const renderCount = useRef(0);
```

```
  renderCount.current += 1;
  let colorInfo = color ? color : "Unknown";

  console.log(`Car ${make} Rendered ${renderCount.current} times`, {
    make,
  });

  useEffect(() => {
    console.log(`Car ${make} Mounted`);
    // Fonction de nettoyage : appelée au démontage (bouton « Unmount Home »)
    return () => {
      console.log(`Car ${make} UnMounted`);
    };
  }, [make]);

  return (
    <div className="car-item">
      <div>{make}</div>
      <div>{colorInfo}</div>
    </div>
  );
}

const allCars = [
  { make: "BMW", color: "Blue" },
  { make: "Audi", color: "White" },
  { make: "Mercedes", color: "Black" },
];

function Cars() {
  const renderCount = useRef(0);
  renderCount.current += 1;

  const [cars, setCars] = useState([]);
  const [loading, setLoading] = useState(true);

  console.log(`Home Rendered ${renderCount.current} times`, {
    loading,
    cars: cars.length,
  });

  // useEffect avec [] : ça tourne une seule fois, au montage
  useEffect(() => {
    console.log("Home Mounted");
    console.log("Fetching cars...");
    setLoading(true);

    // const timerId =
    setTimeout(() => {
      console.log("Cars fetched");
      setCars(allCars);
      setLoading(false);
    }, 1000);
    return () => {
      console.log("Home UnMounted");
    };
  }, []);

  const handleFilter = (selectedMake) => {
    console.log(`Filtering cars by make: ${selectedMake}`);
    if (selectedMake === "ALL") {
      setCars(allCars);
    } else {
      const filteredCars = allCars.filter((car) => car.make === selectedMake);
      setCars(filteredCars);
```

```
      console.log(`Home filtered by make ${selectedMake}`);
    }
  };

  return (
    <div>
      <h1>Welcome to the Home Page</h1>
      <h2>Car List</h2>

      {loading ? (
        <div className="loading">Loading cars...</div>
      ) : (
        <>
          <div className="container">
            {cars.map((car, index) => (
              <Car key={index} make={car.make} color={car.color} />
            ))}
          </div>
          <button onClick={() => handleFilter("BMW")}>Set Make to BMW</button>
        </>
      )}
    </div>
  );
}

export default Cars;
```

## Étape 1 → 2 etape-1-cycle-logs → etape-2-react-memo

React.memo et le state seeMore pour observer les renders inutiles

MODIFIÉ **src/components/Cars.jsx** +13 −2

```
@@ -1,3 +1,4 @@
+ import React from "react";
  import { useEffect, useRef, useState } from "react";

  function Car({ make, color }) {
@@ -26,6 +27,9 @@
    );
  }

+ // React.memo : Car ne se re-rend que si ses props changent
+ const MemoizedCar = React.memo(Car);
+
  const allCars = [
    { make: "BMW", color: "Blue" },
    { make: "Audi", color: "White" },
@@ -38,6 +42,7 @@

    const [cars, setCars] = useState([]);
    const [loading, setLoading] = useState(true);
+   const [seeMore, setSeeMore] = useState(false);

    console.log(`Home Rendered ${renderCount.current} times`, {
      loading,
@@ -74,7 +79,7 @@

    return (
      <div>
-       <h1>Welcome to the Home Page</h1>
+       <h1>{"Welcome to the Home Page"}</h1>
        <h2>Car List</h2>

        {loading ? (
@@ -83,10 +88,16 @@
          <>
            <div className="container">
              {cars.map((car, index) => (
-               <Car key={index} make={car.make} color={car.color} />
+               <MemoizedCar key={index} make={car.make} color={car.color} />
              ))}
            </div>
            <button onClick={() => handleFilter("BMW")}>Set Make to BMW</button>
+           <button
+             style={{ margin: "10px" }}
+             onClick={() => setSeeMore(!seeMore)}
+           >
+             See More
+           </button>
          </>
        )}
      </div>
```

## Étape 2 → 3 etape-2-react-memo → etape-3-props-recreees

On passe style et onSelect à Car, recréés à chaque render : BMW re-render de nouveau, React.memo ne suffit plus

MODIFIÉ **src/components/Cars.jsx** +34 −11

```
@@ -1,34 +1,38 @@
 import React from "react";
 import { useEffect, useRef, useState } from "react";

-function Car({ make, color }) {
+function Car({ make, color, onSelect, style }) {
   // useRef garde le compteur d'un render à l'autre, sans provoquer de nouveau render
   const renderCount = useRef(0);
   renderCount.current += 1;
-  let colorInfo = color ? color : "Unknown";

-  console.log(`Car ${make} Rendered ${renderCount.current} times`, {
-    make,
-  });
+  console.log(`Car ${make} Rendered ${renderCount.current} times`);

   useEffect(() => {
     console.log(`Car ${make} Mounted`);
     // Fonction de nettoyage : appelée au démontage (bouton « Unmount Home »)
-    return () => {
-      console.log(`Car ${make} UnMounted`);
-    };
+    return () => console.log(`Car ${make} UnMounted`);
   }, [make]);

   return (
-    <div className="car-item">
+    <div className="car-item" style={style} onClick={() => onSelect(make)}>
       <div>{make}</div>
-      <div>{colorInfo}</div>
+      <div>{color ?? "Unknown"}</div>
     </div>
   );
 }

 // React.memo : Car ne se re-rend que si ses props changent
 const MemoizedCar = React.memo(Car);
+
+/*
+const MemoizedCar = React.memo(Car, (prevProps, nextProps) => {
+  return (
+    prevProps.brand === nextProps.brand &&
+    prevProps.color === nextProps.color
+  );
+});
+*/

 const allCars = [
   { make: "BMW", color: "Blue" },
@@ -48,6 +52,19 @@
     loading,
     cars: cars.length,
   });
+
+  // Sans useCallback : une NOUVELLE fonction à chaque render de Home → React.memo ne sert plus à rien
```

```
      57  +   const onSelect = (make) => {
      58  +     console.log("Selected:", make);
      59  +   };
      60  +
      61  +   // Sans useMemo : un NOUVEL objet à chaque render de Home → même problème
      62  +   const carStyle = {
      63  +     padding: "10px",
      64  +     marginBottom: "10px",
      65  +     border: "1px solid #ccc",
      66  +     cursor: "pointer",
      67  +   };
51    68
52    69      // useEffect avec [] : ça tourne une seule fois, au montage
53    70      useEffect(() => {
          @@ -88,7 +105,13 @@
88   105              <>
89   106                <div className="container">
90   107                  {cars.map((car, index) => (
91       -                  <MemoizedCar key={index} make={car.make} color={car.color} />
     108  +                  <MemoizedCar
     109  +                    key={index}
     110  +                    make={car.make}
     111  +                    color={car.color}
     112  +                    onSelect={onSelect}
     113  +                    style={carStyle}
     114  +                  />
92   115                  ))}
93   116                </div>
94   117                <button onClick={() => handleFilter("BMW")}>Set Make to BMW</button>
```

## Étape 3 → 4 etape-3-props-recreees → etape-4-usecallback-usememo

useCallback (onSelect) et useMemo (carStyle) stabilisent les props : BMW ne re-render plus

MODIFIÉ **src/components/Cars.jsx** +14 −11

```
@@ -1,4 +1,4 @@
- import React from "react";
+ import React, { useCallback, useMemo } from "react";
  import { useEffect, useRef, useState } from "react";

  function Car({ make, color, onSelect, style }) {
@@ -53,18 +53,21 @@
      cars: cars.length,
    });

-   // Sans useCallback : une NOUVELLE fonction à chaque render de Home → React.memo ne sert plus à rien
-   const onSelect = (make) => {
+   // useCallback: même référence entre les renders
+   const onSelect = useCallback((make) => {
      console.log("Selected:", make);
-   };
+   }, []);

-   // Sans useMemo : un NOUVEL objet à chaque render de Home → même problème
-   const carStyle = {
-     padding: "10px",
-     marginBottom: "10px",
-     border: "1px solid #ccc",
-     cursor: "pointer",
-   };
+   // useMemo: même objet entre les renders
+   const carStyle = useMemo(
+     () => ({
+       padding: "10px",
+       marginBottom: "10px",
+       border: "1px solid #ccc",
+       cursor: "pointer",
+     }),
+     []
+   );

    // useEffect avec [] : ça tourne une seule fois, au montage
    useEffect(() => {
```